UTEC
Universidad Tecnológica

Patrón Adapter

El **Patrón Adapter** es un patrón de diseño estructural que permite que dos clases con interfaces incompatibles trabajen juntas. Este patrón actúa como un intermediario o "adaptador" que convierte la interfaz de una clase en otra interfaz que el cliente espera. Es especialmente útil cuando se necesita utilizar una clase existente, pero su interfaz no coincide con lo que se requiere en el contexto actual.

Objetivo

El objetivo principal del Patrón Adapter es facilitar la interoperabilidad entre clases que, de otra manera, no podrían trabajar juntas debido a diferencias en sus interfaces. Esto permite que se puedan reutilizar clases existentes sin necesidad de modificar su código, respetando así el principio de abierto/cerrado (Open/Closed Principle) de SOLID.

Cómo Funciona

1. **Cliente**: La clase que necesita utilizar la funcionalidad de otra clase con una interfaz incompatible.
2. **Interfaz Objetivo (Target)**: La interfaz que el cliente espera.
3. **Adaptado (Adaptee)**: La clase con una interfaz incompatible que se desea utilizar.
4. **Adaptador (Adapter)**: Una clase que implementa la interfaz objetivo y se comunica con el adaptado, transformando las llamadas del cliente en algo que el adaptado pueda entender.

Tipos de Adapter

- **Adapter de Clase**: Utiliza herencia para adaptar una clase a la interfaz objetivo.
- **Adapter de Objeto**: Utiliza composición para adaptar una clase a la interfaz objetivo. Este es el más común en Java, ya que permite mayor flexibilidad.

Ejemplo en Java: Adaptación de un Enchufe

Supongamos que tenemos un cliente que espera una interfaz `EnchufeEuropeo`, pero contamos con una clase existente `EnchufeAmericano` que tiene una interfaz diferente.

1. Interfaz `EnchufeEuropeo`

```
interface EnchufeEuropeo {
    void conectar();
```

UTEC
Universidad Tecnológica

```
}
```

2. Clase `EnchufeAmericano`

```
class EnchufeAmericano {
    public void plugIn() {
        System.out.println("Enchufe americano conectado.");
    }
}
```

3. Adaptador `AdaptadorEnchufe` (Adapter de objeto)

```
class AdaptadorEnchufe implements EnchufeEuropeo {
    private EnchufeAmericano enchufeAmericano;

    public AdaptadorEnchufe(EnchufeAmericano enchufeAmericano) {
        this.enchufeAmericano = enchufeAmericano;
    }

    @Override
    public void conectar() {
        enchufeAmericano.plugIn(); // Adaptación de la llamada
    }
}
```

4. Uso del Adaptador

```
public class Main {
    public static void main(String[] args) {
        EnchufeAmericano enchufeAmericano = new EnchufeAmericano();
        EnchufeEuropeo enchufeEuropeo = new
AdaptadorEnchufe(enchufeAmericano);

        enchufeEuropeo.conectar(); // Esto conectará el enchufe
americano usando la interfaz europea
    }
}
```

UTEC
Universidad Tecnológica

Resultado:

```
Enchufe americano conectado.
```

Explicación: El cliente (el método `main`) espera trabajar con un `EnchufeEuropeo`, pero gracias al adaptador (`AdaptadorEnchufe`), puede utilizar un `EnchufeAmericano` sin modificar su interfaz.

Usos Comunes del Patrón Adapter

1. **Integración de librerías o sistemas antiguos**: Cuando se necesita utilizar una librería o sistema que no se puede modificar y cuya interfaz es incompatible con el nuevo código.
2. **Compatibilidad de APIs**: Para hacer compatible una nueva API con una existente, adaptando las diferencias entre las interfaces.
3. **Sistemas de legado**: En aplicaciones que han evolucionado con el tiempo, permitiendo que los componentes nuevos se comuniquen con sistemas antiguos.

Ventajas del Patrón Adapter

- **Reutilización de código existente**: Permite usar clases existentes sin necesidad de modificarlas.
- **Facilita la interoperabilidad**: Soluciona problemas de compatibilidad entre clases o componentes.
- **Respetar el principio de abierto/cerrado**: No se necesita modificar las clases originales.

Comparación con Otros Patrones

El patrón Adapter se puede confundir a veces con otros patrones estructurales como **Facade** o **Decorator**. Sin embargo, se diferencia en que **Facade** simplifica una interfaz para el cliente, mientras que **Decorator** agrega comportamiento adicional sin cambiar la interfaz. **Adapter**, por su parte, es específico en convertir una interfaz en otra que el cliente espera.

Conclusión

El Patrón Adapter es una solución eficiente para permitir que clases con interfaces incompatibles trabajen juntas, facilitando la reutilización de código y manteniendo la cohesión en el diseño del software. Es especialmente útil en sistemas que requieren

integración con código legado o librerías de terceros, haciendo que el proceso de adaptación sea transparente para el cliente.

Principio de Abierto/Cerrado (Open/Closed Principle) de SOLID

El Principio de Abierto/Cerrado es uno de los cinco principios fundamentales del diseño de software conocidos como SOLID. Este principio establece que:

"Las clases, módulos, funciones u otras entidades de software deben estar abiertas para la extensión, pero cerradas para la modificación."

¿Qué significa esto?

- **Abierto para la extensión**: Esto significa que el comportamiento de una entidad de software debe poder extenderse para cubrir nuevos requisitos o funcionalidades.
- **Cerrado para la modificación**: Significa que la estructura existente del código no debe ser alterada una vez que ha sido desarrollada y probada, para evitar introducir errores o inconsistencias.

Ejemplo Simple en Java

Supongamos que tienes una clase `Empleado` que calcula el salario de diferentes tipos de empleados (por hora, por salario fijo, etc.).

Versión sin seguir el principio (código rígido):

```
class Empleado {
    public double calcularSalario(String tipoEmpleado, double
horasTrabajadas, double salarioBase) {
        if (tipoEmpleado.equals("hora")) {
            return horasTrabajadas * salarioBase;
        } else if (tipoEmpleado.equals("fijo")) {
            return salarioBase;
        }
        // más tipos de empleados...
        return 0;
    }
}
```

UTEC
Universidad Tecnológica

En este ejemplo, cada vez que se introduce un nuevo tipo de empleado, debemos modificar la clase `Empleado`. Esto viola el principio de abierto/cerrado porque la clase no está cerrada a modificaciones.

Versión siguiendo el principio (código flexible):

```
abstract class Empleado {
    public abstract double calcularSalario();
}

class EmpleadoPorHora extends Empleado {
    private double horasTrabajadas;
    private double salarioHora;

    public EmpleadoPorHora(double horasTrabajadas, double salarioHora) {
        this.horasTrabajadas = horasTrabajadas;
        this.salarioHora = salarioHora;
    }

    @Override
    public double calcularSalario() {
        return horasTrabajadas * salarioHora;
    }
}

class EmpleadoFijo extends Empleado {
    private double salarioFijo;

    public EmpleadoFijo(double salarioFijo) {
        this.salarioFijo = salarioFijo;
    }

    @Override
    public double calcularSalario() {
        return salarioFijo;
    }
}

// Ahora puedes agregar nuevos tipos de empleados sin modificar la clase
Empleado:
class EmpleadoComision extends Empleado {
```

UTEC
Universidad Tecnológica

```
    private double ventas;
    private double porcentajeComision;

    public EmpleadoComision(double ventas, double porcentajeComision) {
        this.ventas = ventas;
        this.porcentajeComision = porcentajeComision;
    }

    @Override
    public double calcularSalario() {
        return ventas * porcentajeComision;
    }
}
```

En este segundo ejemplo, hemos diseñado la clase `Empleado` para que sea **abierta para la extensión** (puedes crear nuevos tipos de empleados extendiendo la clase base) pero **cerrada para la modificación** (no necesitas cambiar el código existente para agregar nuevas funcionalidades).

Beneficios del Principio de Abierto/Cerrado

1. **Mantenibilidad**: El código es más fácil de mantener porque no necesitas modificar el código existente, lo que reduce el riesgo de introducir nuevos errores.
2. **Reusabilidad**: Las entidades que siguen este principio tienden a ser más reutilizables, ya que están diseñadas para ser extendidas sin alterar su comportamiento básico.
3. **Escalabilidad**: Es más fácil agregar nuevas funcionalidades o requisitos sin afectar el comportamiento existente.